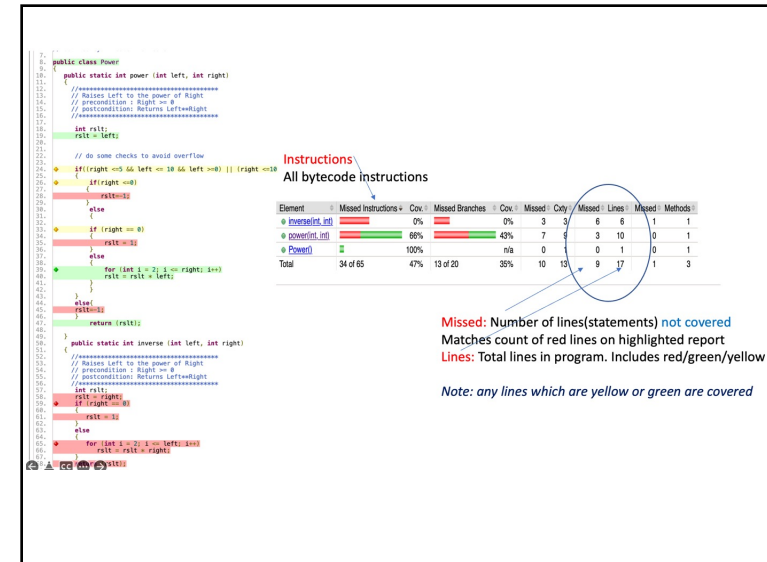


COMS 4170 Software Testing Spring 2026

Chapter 5 – Coverage

1



2

What is "Coverage"

- Common terms heard:
 - Complete testing
 - Exhaustive testing
 - Full coverage
- But what does this mean?
 - complete what?,
 - exhaustive?
 - full?

3

Impossible in Real Software

- Need to define formally what we mean by coverage
- Coverage criteria tell us "when to stop", or "how good is our test suite"
- We can connect these criteria with test requirements and models

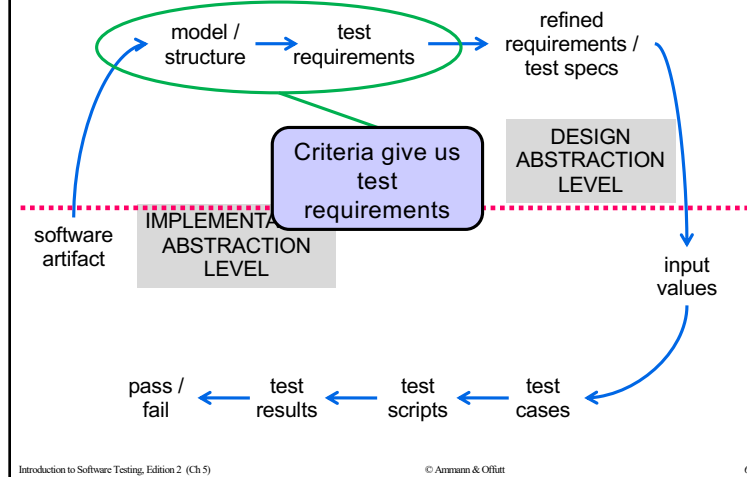
4

Definition

- Test requirement: A *specific element* of a software artifact that a test case must satisfy or cover
- Test requirements usually come in sets (TR) – denotes a set of test requirements
- Can apply to source code, design components, specifications, modeling elements (e.g. graphs)

5

Model-Driven Test Design



6

New : Test Coverage Criteria

A tester's job is simple : Define a model of the software, then find ways to cover it



Test Requirements : A specific element of a software artifact that a test case must satisfy or cover



Coverage Criterion : A rule or collection of rules that impose test requirements on a test set

Testing researchers have defined dozens of criteria, but they are all really just a few criteria on four types of structures ...

Introduction to Software Testing, Edition 2 (Ch 5)

© Ammann & Offutt

7

7

Source of Structures

- These structures can be extracted from lots of software artifacts
 - Graphs can be extracted from UML use cases, finite state machines, source code, ...
 - Logical expressions can be extracted from decisions in program source, guards on transitions, conditionals in use cases, ...

Introduction to Software Testing, Edition 2 (Ch 5)

8

8

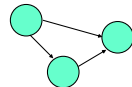
Criteria Based on Structures

Structures : Four ways to model software

1. Input Domain Characterization (sets)

A: {0, 1, >1}
B: {600, 700, 800}
C: {swe, cs, isa, infs}

2. Graphs



3. Logical Expressions

(not X or not Y) and A and B

4. Syntactic Structures (grammars)

```
if (x > y)
  z = x - y;
else
  z = 2 * x;
```

Introduction to Software Testing, Edition 2 (Ch 5)

© Ammann & Offutt

9

Example : Jelly Bean Coverage

Flavors :

1. Lemon
2. Pistachio
3. Cantaloupe
4. Pear
5. Tangerine
6. Apricot

Colors :

1. Yellow (Lemon, Apricot)
2. Green (Pistachio)
3. Orange (Cantaloupe, Tangerine)
4. White (Pear)



Possible coverage criteria :

1. Taste one jelly bean of **each flavor**
 - Deciding if **yellow jelly bean** is **Lemon or Apricot** is a controllability problem
2. Taste one jelly bean of **each color**

Introduction to Software Testing, Edition 2 (Ch 5)

© Ammann & Offutt

10

10

Coverage

Given a set of test requirements TR for coverage criterion C , a test set T satisfies C coverage if and only if for every test requirement tr in TR , there is at least one test t in T such that t satisfies tr

- **Infeasible test requirements** : *test requirements that cannot be satisfied*
 - No test case values exist that meet the test requirements
 - Example: Dead code
 - Detection of infeasible test requirements is formally undecidable for most test criteria
- Thus, 100% coverage is **impossible** in practice

Introduction to Software Testing, Edition 2 (Ch 5)

© Ammann & Offutt

11

11

More Jelly Beans

$T1 = \{ \text{three Lemons, one Pistachio, two Cantaloupes, one Pear, one Tangerine, four Apricots} \}$

- Does test set T1 satisfy the flavor criterion ?

$T2 = \{ \text{One Lemon, two Pistachios, one Pear, three Tangerines} \}$

- Does test set T2 satisfy the flavor criterion?
- Does test set T2 satisfy the color criterion?

Introduction to Software Testing, Edition 2 (Ch 5)

© Ammann & Offutt

12

12

Back to Software

- Cover all **statements**
- Cover all **branches** in the program
 - This leads to two tests for each decision (true and false)
- **Method coverage**
 - Call each method at least once

13

Definition

- Coverage Criterion:
 - *A rule or collection of rules that impose test requirements on a test set*
- Coverage:
 - Given a set of test requirements, *TR*, for a coverage criterion *C*, a test set satisfies *C*, if and only if, *for every test requirement, tr, in TR, at least one test, t, in TR exists that satisfies tr*

14

Examples from Class

- **TR**
 - Cover all statements in a program
- **Approach**
 - Use Jacoco. Keep adding tests until all statements covered
 - What if we can't cover all statements?
 - We can measure the percentage covered – this gives us a quality metric

15

Statement Coverage with Jacoco

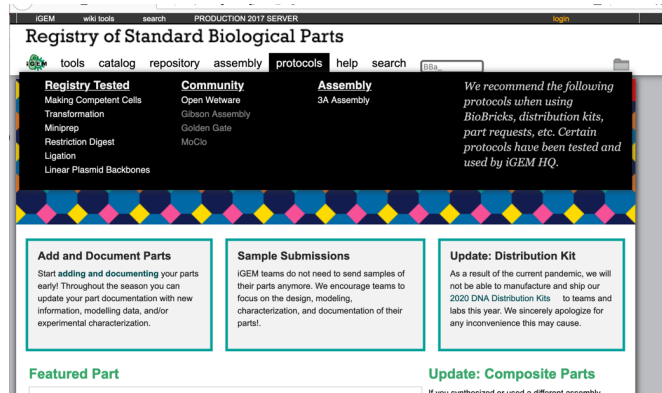
JaCoCo
Maven plug-in example with Offline Instrumentation
>
default

default

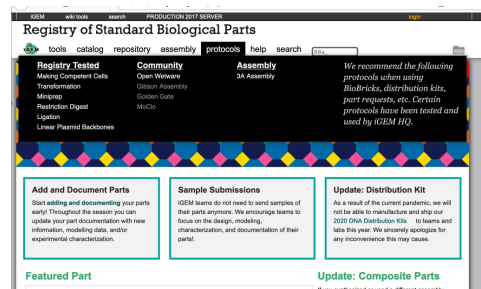
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TriangleType	93%		81%		5	13	1	10	1	2	0	1
Triangle	100%		n/a		0	1	0	2	0	1	0	1
Total	3 of 93	96%	4 of 22	81%	5	14	1	12	1	3	0	2

16

Web App

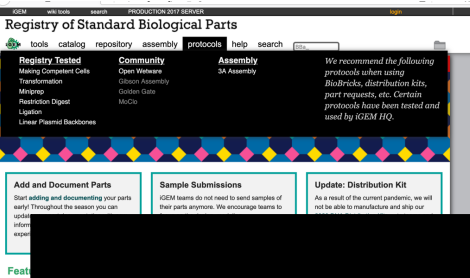


17



- Cover all menu items
- Cover all edges (between menu items)
- Cover all links

18



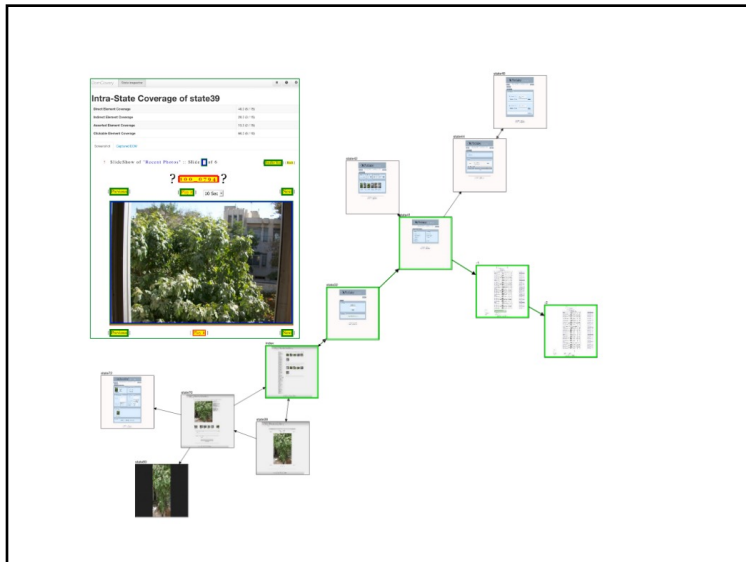
Can use a graph to show structure of web page elements

-
-
-

19



20



21

System vs. Unit Testing

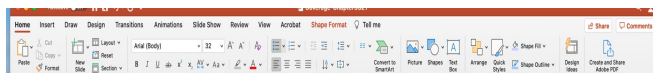
- We have focused a lot on **unit testing**
- However, system testing is also necessary
- For **system testing** we need to think about the **external view of the program**
- Often don't have access to code

22

Break Out

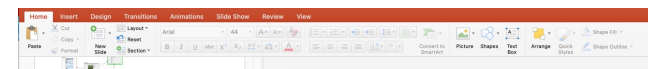
Open PowerPoint and discuss some strategies to

- Cover **functionality** using **menus**
- **What elements** should you test?
- How might you **represent** this application?
- When we return we will discuss – Next week will talk about some GUI Testing strategies



23

Discussion of PowerPoint



Break Out

Open PowerPoint and discuss some strategies to

- Cover functionality using menus
- What elements should you test?
- How might you represent this application?
- When we return we will discuss – on Tuesday will take about some GUI Testing strategies



25

Definition

- **Minimal Test Set**
- Given TR and a test set T that satisfies all test requirements, T is minimal if **removing any single test from T will cause T to no longer satisfy** any all test requirements

26

Definition

- **Minimum Test Set**
- Given TR and a test set T that satisfies all test requirements, T is minimum if there is **no smaller set of tests** that also satisfy all test requirements

27

Comment

- Finding a **minimal test suite** is **normally straight forward**—delete tests one by one and evaluate
- Finding a **minimum test suite** is **much harder** and usually undecidable

28

Definition

- **Coverage Level**
 - Give a set of requirements TR and a test set T , the coverage level is the **ratio of the number of test requirements satisfied by T to the size of TR**

29

JaCoCo Maven plug-in example with Offline Instrumentation > default

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TriangleType	5	93%	1	81%	5	13	1	10	1	2	0	1
Triangle	0	100%	n/a	0	1	0	2	0	1	0	1	1
Total	3 of 93	96%	4 of 22	81%	5	14	1	12	1	3	0	2

Completeness

- Some requirements **cannot be satisfied**

```
...  
...  
x=y+1;  
if(x <=y){  
    System.out.println("not reachable");  
}
```

30

Infeasible Test Requirements

- **No test case value exists that meets an infeasible test requirement**
- Example:
 - **Dead code**
- Detection of infeasible test requirements **is formally undecidable for most coverage**
 - therefore 100% coverage is **usually impossible** in practice

31

Using Coverage Criterion

1. **Generator:** Used to generate test case values to satisfy this criterion. May be hard without good automated tools
(we did this with EvoSuite)
2. **Recognizer:** Create tests and then measure coverage (by running tests)
(we did this with Jacoco)

32

How much Coverage is good Enough?

- Since we can't determine what can/cannot be covered, **how do we know we have covered enough?**
- Sometimes we use the term **"metrics"** to indicate that these are just measures of how much is covered

33

Caveat

- The set TR depends on the artifact we are testing (system under test)
- Jelly bean example:
 - Requirement color=purple makes no sense since the factory does not make purple jelly beans
- Code:
 - Execute line 42 – only meaningful if program has a line 42

34

Subsumption

- We can often compare coverage criteria to each other
- Jelly Beans
- Flavor criterion
 - Requires each flavor tested once
 - Color criterion
 - Try each color once

Flavor subsumes color!

35

Definition

- *Criteria Subsumption:*
 - A coverage criterion C_1 subsumes C_2 if and only if every test set that satisfies criterion C_1 also satisfies C_2

This has to hold for every test set, not just some

36

Branch and Statement

- If a test set has covered every branch in a program (100% branch coverage), then the test set is guaranteed to also cover every statement in the program

37

Infeasibility and Subsumption

- Sometimes a criterion, C_1 will subsume another criterion C_2 if **and only if all test requirements are feasible**. If some requirements in C_1 are infeasible, C_1 may not subsume C_2 .
- In practice this usually does not have a huge impact

38

Advantages of Coverage Criterion

- Make testing more efficient and effective
- Testing is expensive/labor intensive. Coverage Criteria help us decide “which tests” to run
- Coverage criteria help to carve out the test space. Helps us sample across the space more effectively

39

Advantages of Coverage Criterion (cont.)

- Provides stopping rules (e.g. 95% statement coverage)
- Can add traceability (we know what we tested and can map back to a model)

40

Issues Impacting Criteria

- Difficulty of computing test requirements
- Difficulty of generating inputs
- How well tests reveal faults

41